Task 1 – 8-Puzzle with A* Search

Documentation

Introduction to Artificial Intelligence and Data Science, WS 26/27
Group members: Dominik Barat, Daniel Rosenthal, Fabian Sefranek, Benjamin Suskic

1. Task description

The objective of this task is to implement an A* search algorithm in Python to solve the 8-puzzle and compare the performance of two heuristic functions: Hamming distance and Manhattan distance.

The 8-puzzle consists of a 3×3 board containing eight numbered tiles and one blank space. A move consists of sliding a tile adjacent to the blank into the empty position, with each move having a cost of 1. The goal is to reach the ordered state

0 1 2
3 4 5
6 7 8

using as few moves as possible.

The implementation generates random start states, checks their solvability, and solves them using A* with both heuristic functions. To evaluate their performance, the same 100 solvable start states are tested with each heuristic. The number of expanded nodes and execution time are recorded, and their mean and standard deviation are calculated.

Finally, the documentation discusses the time and space complexity of the heuristics and compares their performance based on the experimental results.

The program optionally exports the individual measurements to a CSV file for further analysis.

How to run it

The script needs only the Python standard library (Python 3.8 or newer).

Command	Effect
`python puzzle8.py`	100 random puzzles with seed 42, including the example solution
`python puzzle8.py --n 50 --seed 7`	Runs 50 puzzles using a different, reproducible random sequence
`python puzzle8.py --csv results.csv`	Also writes the raw measurements per puzzle

2. Software architecture

The program is organized into seven logical components (modules), each responsible for a specific part of the solution process. The Main component coordinates these components and controls the overall execution.

First, the puzzle model generates the goal state and random starting states. Each starting state is checked for solvability before it is included in the experiment. The experiment then solves each valid starting state twice: once using the Hamming heuristic and once using the Manhattan heuristic.

During each search, A* explores possible puzzle states and uses the selected heuristic to estimate how far each state is from the goal. The algorithm returns a solution path and the number of expanded nodes. The experiment also measures the execution time of each search.

Finally, the statistics component summarizes the collected measurements by calculating the mean and standard deviation for each heuristic. The results are printed in a report and can optionally be exported to a CSV file.

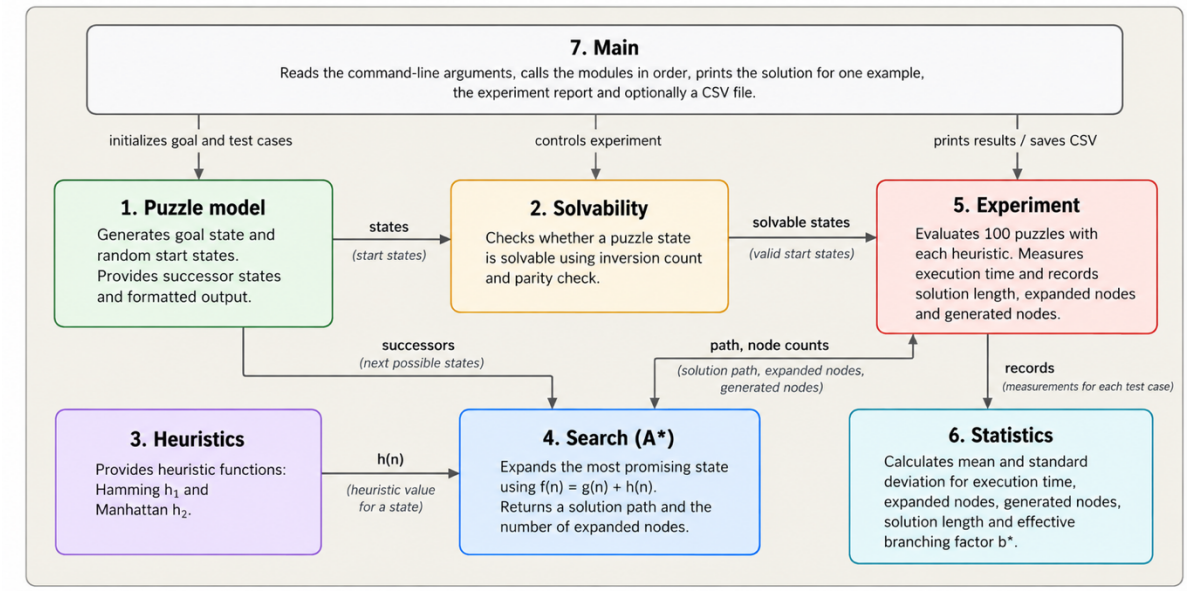


Figure 1: Components of puzzle8.py and the data passed between them

3. Modules and interfaces

The implementation is contained in a single Python file, puzzle8.py, organized into seven logical components. A puzzle state is represented as a tuple of nine integers with 0 representing the blank space.

Component	Responsibility	Input	Output
1. Puzzle model	Defines the goal state, generates random states and legal successors, and formats puzzle states.	Puzzle state, random generator	Goal state, start states, successor states, formatted output
2. Solvability	Decides whether the goal is reachable.	Start state, goal state	Boolean (True or False)

Component	Responsibility	Input	Output
3. Heuristics	Estimates the remaining distance to the goal using Hamming or Manhattan distance.	Goal state to initialize the heuristic. Puzzle state for evaluation	Integer estimate h(n)
4. Search (A*)	Finds an optimal solution using the selected heuristic.	Start state, goal state, heuristic function	Solution path, number of moves, node counts
5. Experiment	Runs both heuristics on the same solvable start states and measures performance.	Start states, goal state, heuristic functions	Individual measurements for each run
6. Statistics	Summarizes the measurements.	Experiment records	Mean and standard deviation per measure, effective branching factor
7. Main	Processes command-line arguments, coordinates execution and displays results.	Command-line arguments	Console output, optional CSV file

The components interact through function calls. For example, the A* search receives a heuristic function as a parameter, allowing both heuristics to use the same search implementation. The experiment collects the search results, which are then processed by the statistics component.

Example output of a solution

For the example state the script prints every move and the resulting board. The first two and the last of 23 steps with seed 42:

```
Start:
2 5 7
8 6 1
4 _ 3

Step 1 of 23: tile 6 moves down
2 5 7
8 _ 1
4 6 3

Step 2 of 23: tile 8 moves right
2 5 7
_ 8 1
4 6 3
...
Step 23 of 23: tile 1 moves right
_ 1 2
3 4 5
6 7 8

Goal reached after 23 moves.
```

4. Design decisions

The implementation was designed to make the search efficient, the comparison between the two heuristics fair, and the results easy to verify. The following sections explain the main design decisions and why they were made.

1. Representing puzzle states as tuples.

Each puzzle state is stored as a tuple of nine integers instead of a two-dimensional list. For example, the goal state is represented as (0, 1, 2, 3, 4, 5, 6, 7, 8). The tuple represents the board row by row, with 0 representing the blank space. Tuples cannot be changed after creation and can be used as keys in dictionaries or stored in sets. This makes it easier for A* to recognize states that have already been encountered and avoid unnecessary repeated work.

2. Precomputing frequently used information.

Some information is needed repeatedly during the search. For example, the program needs to know which positions the blank can move to and how far each tile is from its goal position. Instead of calculating this information every time, the program stores it in tables before the search begins. The algorithm can then look up the required information quickly, reducing repeated calculations.

3. Checking solvability before searching.

Not every possible arrangement of the puzzle can reach the goal state. The program therefore counts *inversions*, which occur when a larger numbered tile appears before a smaller one in the row-by-row sequence. For a 3×3 puzzle, the parity (whether the count is even or odd) determines whether the state can reach the configured goal. The program compares the inversion parity of the start state with that of the goal state. This check prevents the search from wasting time on impossible puzzles.

4. Generating random but solvable test cases.

To create test cases, the program randomly shuffles the nine puzzle elements and checks whether each resulting state is solvable. Unsolvable states are discarded until the required number of solvable states has been collected. This approach allows the experiment to use randomly selected solvable states rather than only states created by moving a few steps away from the goal. It also ensures that both heuristics are tested on the same set of puzzles.

5. Using two interchangeable heuristic functions.

The program implements Hamming distance and Manhattan distance to estimate how far a puzzle state is from the goal. Hamming distance counts the numbered tiles that are in the wrong position. Manhattan distance adds up how many horizontal and vertical steps each numbered tile is away from its goal position.

Both heuristics are implemented as separate functions and passed to the same A* algorithm. This means the search algorithm does not need to be rewritten to test a different heuristic. The two functions also allow the program to compare how different estimates affect search performance.

6. Choosing A and avoiding unnecessary repeated searches.*

A* evaluates each state using (f(n) = g(n) + h(n)). Here, (g(n)) is the number of moves made so far, while (h(n)) estimates the number of moves remaining to reach the goal. The algorithm prioritizes states with the lowest estimated total cost.

Both implemented heuristics are *admissible*, meaning they never estimate a remaining cost higher than the true minimum. Consequently, A* can find an optimal solution with either heuristic. Manhattan distance also dominates Hamming distance because its estimate is always at least as large and is often more informative. This generally helps A* explore fewer states.

To manage the search efficiently, the program uses a priority queue to select the next state and stores the best known path cost for each encountered state. A separate set tracks states that have already been expanded. These structures help avoid expanding the same state repeatedly when it does not need to be revisited.

7. Ensuring a fair and reproducible comparison.

Both heuristics are tested on the same 100 solvable start states. This is important because otherwise differences in difficulty between randomly generated puzzles could influence the results. A fixed random seed makes the sequence of generated states reproducible when the same program and settings are used again.

The program measures execution time using Python's time.perf_counter(). It also records expanded nodes, generated nodes and states stored in memory. These measurements provide different perspectives on search effort: execution time measures how long a search takes, while node counts help explain how much work the algorithm performs.

8. Verifying and presenting the results.

The program includes a self-check that compares the solution lengths returned by the two heuristics for each puzzle. Since both heuristics are admissible, the solutions should have the same optimal length. Any difference is therefore a reason to investigate the implementation.

In addition, the program prints the solution to one example puzzle step by step, including the board after each move. This makes it possible to inspect the solution and verify that the goal state is reached. The individual measurements can also be exported to a CSV file for further analysis.

5. Results and complexity

Manhattan needs about one sixteenth of the nodes and one twentieth of the run time of Hamming, at identical solution lengths.

Complexity estimate

The 8-puzzle has 9! = 362,880 arrangements, of which half, 181,440, can be reached from a given state. The blank has 2, 3 or 4 possible moves, so the branching factor b is about 3. The hardest instances need 31 moves; a random instance needs about 22.

Quantity	Estimate
Uninformed tree search to depth 22	About 3^22 ≈ 3.1 × 10^10 nodes
A* time, worst case	O(b^d), exponential in the solution length d
A* space, worst case	O(b^d), because every generated node is kept in memory
A* with a heuristic	About (b*)^d nodes, where b* is the effective branching factor
Upper bound with duplicate detection	181,440 states, the size of the reachable state space
Work per expanded node	At most 4 successors, each with one heuristic call (9 look-ups) and one heap push, O(log n)
Solvability check	28 comparisons, constant

A better heuristic does not change the exponential form. It lowers the base b*, and because the exponent is around 22, a small change of the base has a large effect.

Measured comparison

Values are mean ± sample standard deviation over 100 random solvable puzzles (seed 42). Each puzzle was solved once with each heuristic.

Measure	Hamming	Manhattan
Expanded nodes (memory effort)	13,316 ± 12,942	835 ± 831
Generated nodes	19,753 ± 18,326	1,334 ± 1,308
States stored in memory	19,668 ± 18,226	1,308 ± 1,279
Run time [ms]	75.97 ± 85.06	3.77 ± 4.46
Solution length [moves]	22.47 ± 3.41	22.47 ± 3.41
Effective branching factor b*	1.43 ± 0.02	1.25 ± 0.04

To collect 100 solvable puzzles, 92 unsolvable random states were discarded, which fits the expected share of one half. In all 100 puzzles both heuristics returned the same solution length.

The run times were measured in a Linux cloud container with Python 3 and depend on the computer. The node counts are the same on every computer for the same seed.

Reading the numbers

- **Space.** Hamming expands 15.9 times as many nodes as Manhattan and keeps about 15 times as many states in memory.
- **Time.** Hamming takes 20.1 times as long. The time per expanded node is similar, about 5.7 µs against 4.5 µs, so run time follows the node count.
- **Spread.** The standard deviation is about as large as the mean for nodes and time. The effort grows exponentially with the solution length, so a few long puzzles dominate.
- **Branching factor.** b* drops from 1.43 to 1.25. Both are far below the raw branching factor of about 3.

6. Discussion and conclusions

The choice of heuristic matters more than any detail of the implementation: with the same A* code, Manhattan solves a random 8-puzzle in about 4 ms and Hamming in about 76 ms.

Conclusions

- Both heuristics are admissible and always found a shortest solution. The quality of the result is the same; only the effort differs.
- Manhattan dominates Hamming. It is never smaller and usually much larger (19 against 8 in the example state), so it prunes more of the search tree.
- The better heuristic costs nothing extra here. With the precomputed distance table, both need 9 look-ups per state.

- Memory is the real limit of A*, because all generated nodes stay in memory. For the 8-puzzle this is harmless, as there are only 181,440 reachable states.

Experience

- The solvability check is essential. About half of the random states are unsolvable, and without the check A* would search the whole reachable state space before stopping.
- The duplicate check has to compare path costs. A state can be reached again on a cheaper path, and ignoring it would break optimality.
- Comparing the solution lengths of both heuristics was a cheap and effective test. Any mistake in the search or in a heuristic shows up as a mismatch.
- Mean values alone are misleading. The standard deviation is as large as the mean, so the raw data per puzzle (option `--csv`) is needed for a fair picture.
- Printing the solution move by move made it easy to check a result by hand.
- Setting up the tools took longer than expected. VS Code opened the single script in Restricted Mode, which hides the run button; the script only started after we opened the project folder and trusted it.
- Our first Git commands failed with "not a git repository" because the terminal was in the parent folder of the cloned repository. Opening the repository folder itself in VS Code solved this for the whole group.
- Working with four people in one private repository went well once we agreed to pull before starting and to push small commits.
- The size of the difference between the two heuristics surprised us. We expected Manhattan to be better, but not by a factor of about 16 in expanded nodes.
- The numbers alone were not convincing at first. Seeing the boards change step by step until the goal appeared helped us to understand what A* returns.

Possible improvements

Improvement	Expected effect
Linear conflict added to Manhattan	Stronger admissible heuristic, fewer expanded nodes
Pattern database heuristic	Much stronger estimate, at the cost of precomputation and memory
IDA* instead of A*	Memory linear in the solution length; needed for the 15-puzzle
Incremental heuristic update	Only the moved tile changes, so h can be updated in constant time
State packed into one integer	Less memory per stored state, faster hashing
Repeated timing per puzzle	Less noise in the run time measurement
Diagrams from the CSV data	Nodes over solution length shows the exponential growth directly
Generalisation to N×N boards	Solvability rule must then include the blank row for even widths